

UNIT-II- Web Applications in ASP.NET

ASP.NET Coding Models

A web consists of controls(Label,Button, Text Box etc) and business logic. we can use ASP.NET coding technique to manage these controls and business logic. ASP.NET 4.0 provides two types of coding technique.

- **Single -File page model.**
- **Code-behind page model.**

(1)-Single -File page model:- In Single -File page model,the HTML markup and the functionality of a web application are implemented in same File.

In single -File coding approach, developers write code directly in the.aspx page of the application.A major drawback of the single-File code model is that writing code in a single file, so it is very difficult to read and manage the web pages.

Example:-

See code (Design) .aspx file:-

```
<%@ Page Language="C#" AutoEventWireup="true" CodeFile="Default.aspx.cs"
Inherits="_Default" %>
```

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

```
<script runat="server">
```

```
void Button1_Click(Object sender, EventArgs e)
```

```
{
```

```
Label1.Text =DateTime.Now.ToString();
```

```
}
```

```
void Button2_Click(Object sender, EventArgs e)
```

```
{
    Response.Redirect("default2.aspx");
}

</script>

<html xmlns="http://www.w3.org/1999/xhtml">

<head runat="server">

    <title>Single File Page Model</title>

</head>

<body>

    <form id="form1" runat="server">

        <div>

            <br />

            &nbsp;&nbsp;&nbsp;<asp:Label id="Label1"

                runat="server"></asp:Label>

                &nbsp;<asp:Button id="Button1"

                    runat="server"

                    onclick="Button1_Click"

                    Text="show system date&time!">

                </asp:Button>

                <br />

                <asp:Button ID="Button2" runat="server" onclick="Button2_Click" Text="Ne>

            </div>

        </form>

    </body>

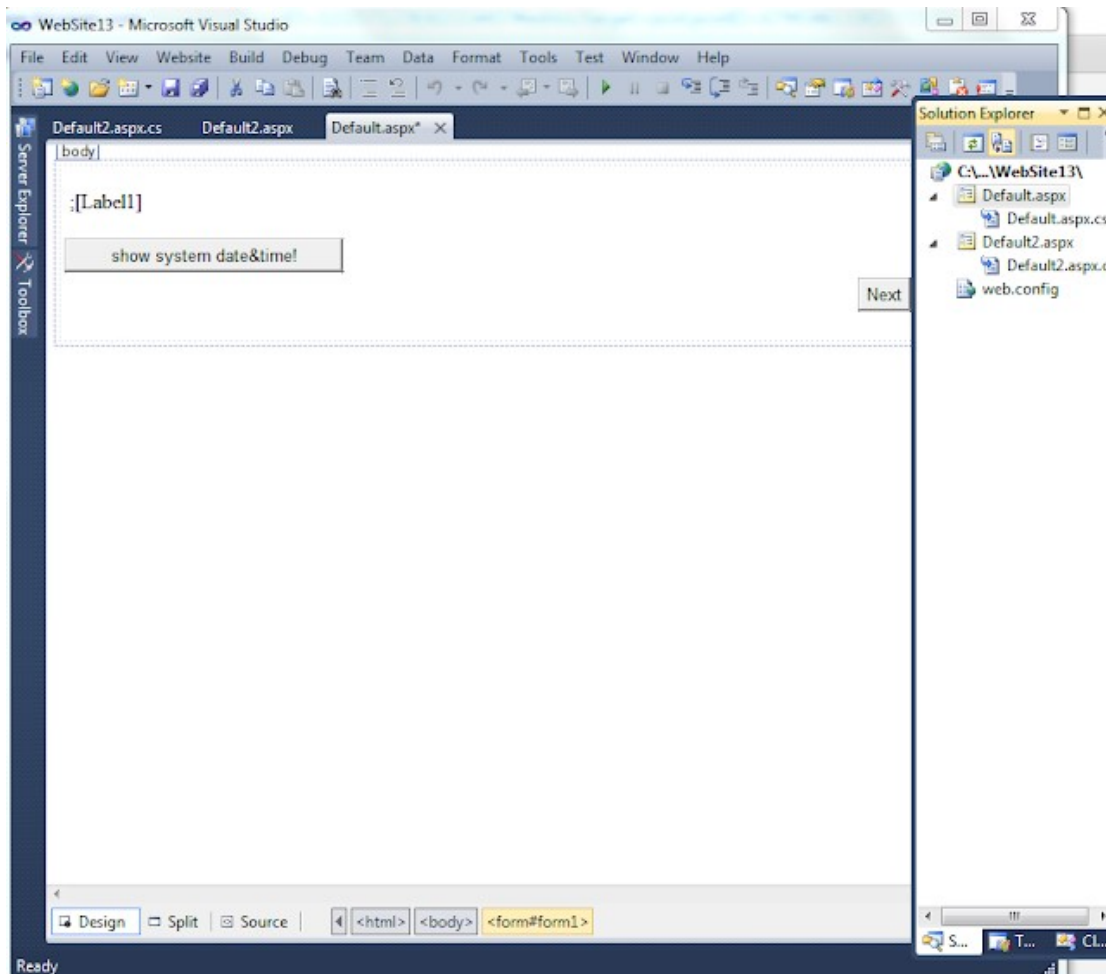
</html>
```

Description:- Here on click on Button1 and Button2 above java script code is called ,and under java script code functionality is executed.

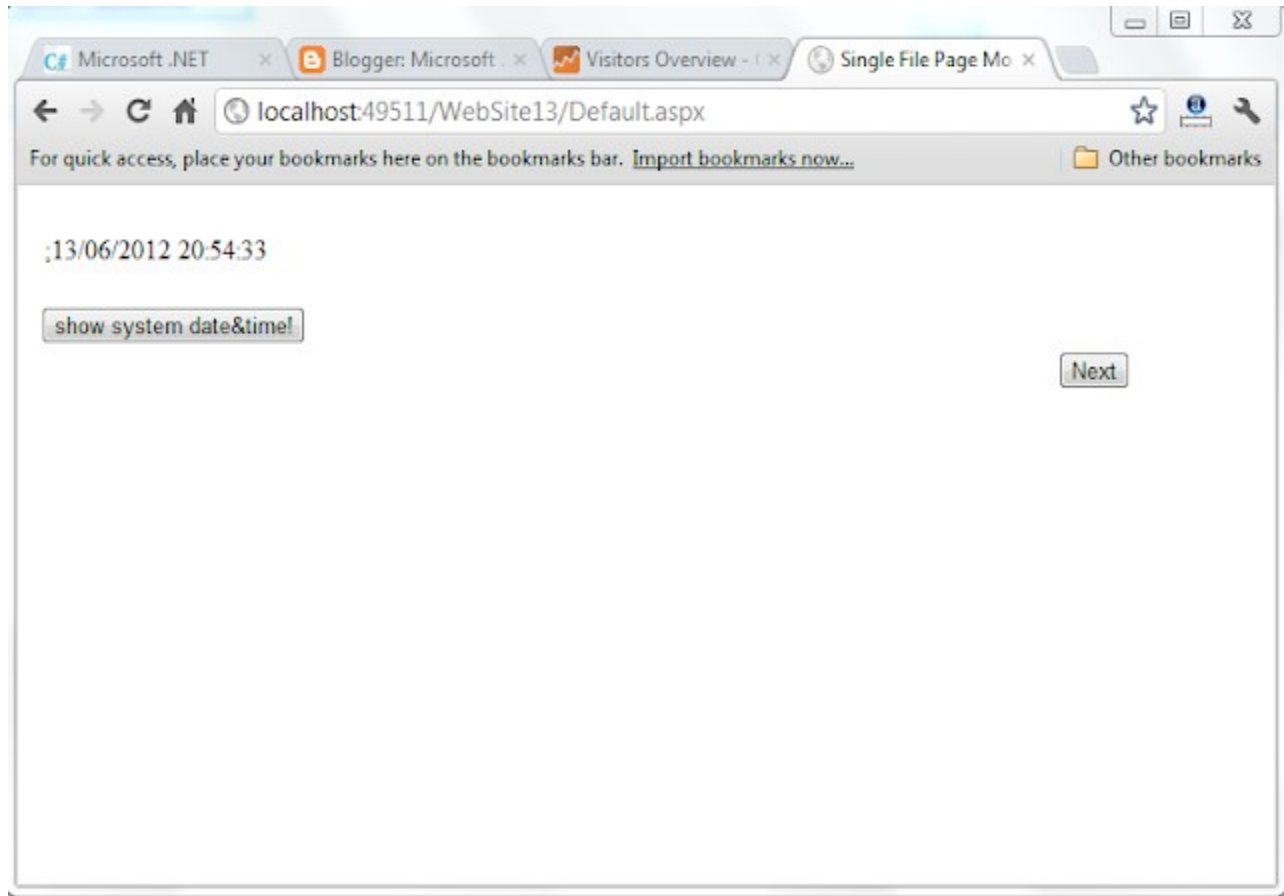
There are some steps to perform this task:-

Steps1:-First open your visual studio->go File-> New website->Select ASP.NET Empty website& c# language->click OK.

Now go Solution Explorer->Add New Item->select Web Form-> Click Add. Drag and drop Label and button control from Toolbox.



Step2:- Now click on Source and paste the above code:Now Run the Default .aspx page.
see Output:-



Explain Advantages of Single Page Model

- In pages where there is not very much code, the convenience of keeping the code and markup in the same file can outweigh other advantages of the code-behind model. For example, it can be easier to study a single-file page because you can see the code and the markup in one place.
- Pages written using the single-file model are slightly easier to deploy or to send to another programmer because there is only one file.
- Because there is no dependency between files, a single-file page is easier to rename if you are using tools other than Visual Studio. (If you use Visual Studio to rename a file, Visual Studio automatically renames both files.)
- Managing files in a source code control system is slightly easier, because the page is self-contained in a single file.

Code -Behind Page Model:- In Code -Behind Page Model,these are Two separate files,Default.aspx and Default.aspx.cs

These two files are linked together to run the web application.All the different version of ASP.NET Support this model, by default.Both the files are combined together during compilation to implement the complete application. In this model, you need to maintain separate code files for each web page, one file stores the code to implement the functionality of web page written in some programming language(c#,vb) and other file stores the HTML Markup of the web application.

Explain Advantages of Code Behind Model:

- Code-behind pages offer a clean separation of the markup (user interface) and code. It is practical to have a designer working on the markup while a programmer writes code.
- Code is not exposed to page designers or others who are working only with the page markup.
- Code can be reused for multiple pages.

Example:-

See the Code for **Default2.aspx** (Design)view:-

```
<%@ Page Language="C#" AutoEventWireup="true" CodeFile="Default2.aspx.cs"
Inherits="_Default" %>
```

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

```
<html xmlns="http://www.w3.org/1999/xhtml">
```

```
<head runat="server">
```

```
<title>Code Behind Page Model</title>
```

```
</head>
```

```
<body>
```

```
<form id="form1" runat="server">
```

```
<div>
```

```
<asp:Label id="Label1"
```

```
runat="server" ></asp:Label>
```

```
 <asp:Button id="Button1"
    runat="server"
    onclick="Button1_Click"
    Text="show system date &time" >
</asp:Button>
</div>
</form>
</body>
</html>
```

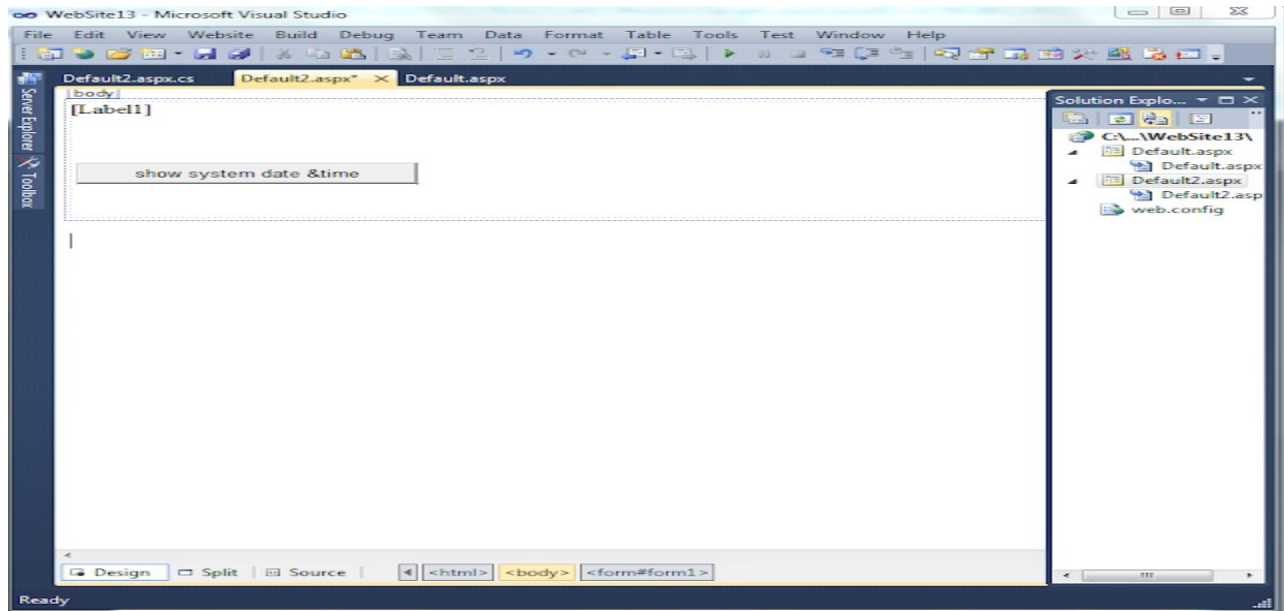
See the code for **Default2.aspx.cs** file:-

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

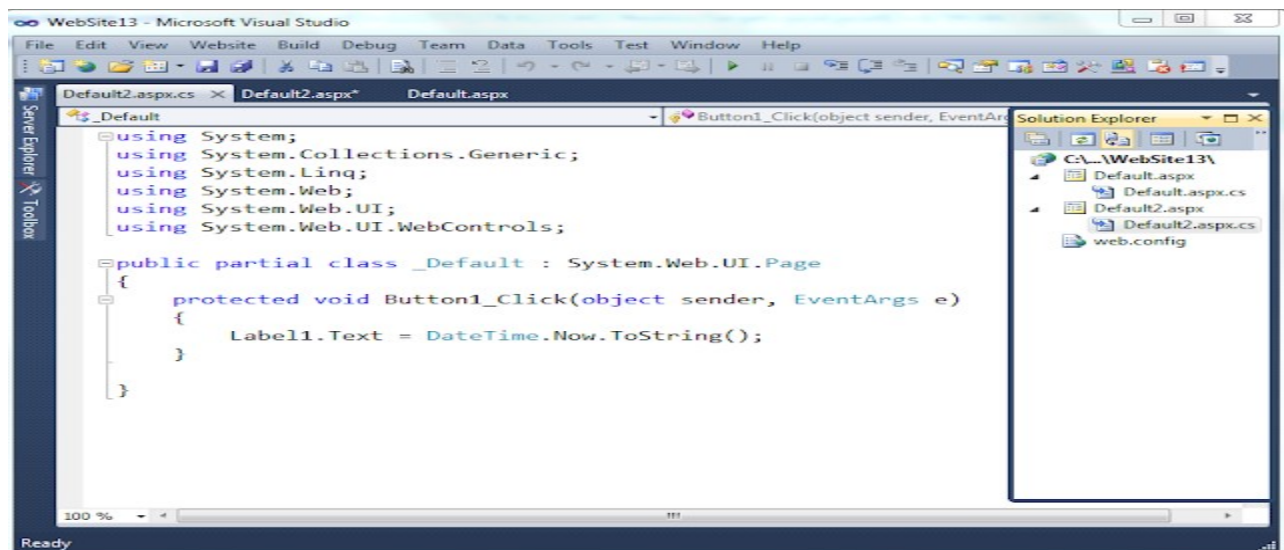
public partial class _Default : System.Web.UI.Page
{
    protected void Button1_Click(object sender, EventArgs e)
    {
        Label1.Text = DateTime.Now.ToString();
    }
}
```

There are some steps to implement this concept:-

Step1:- again open solution Explorer->add new item->Select Web Form->Click Add. Add controls from Toolbox which is given Default2.aspx. see it:-

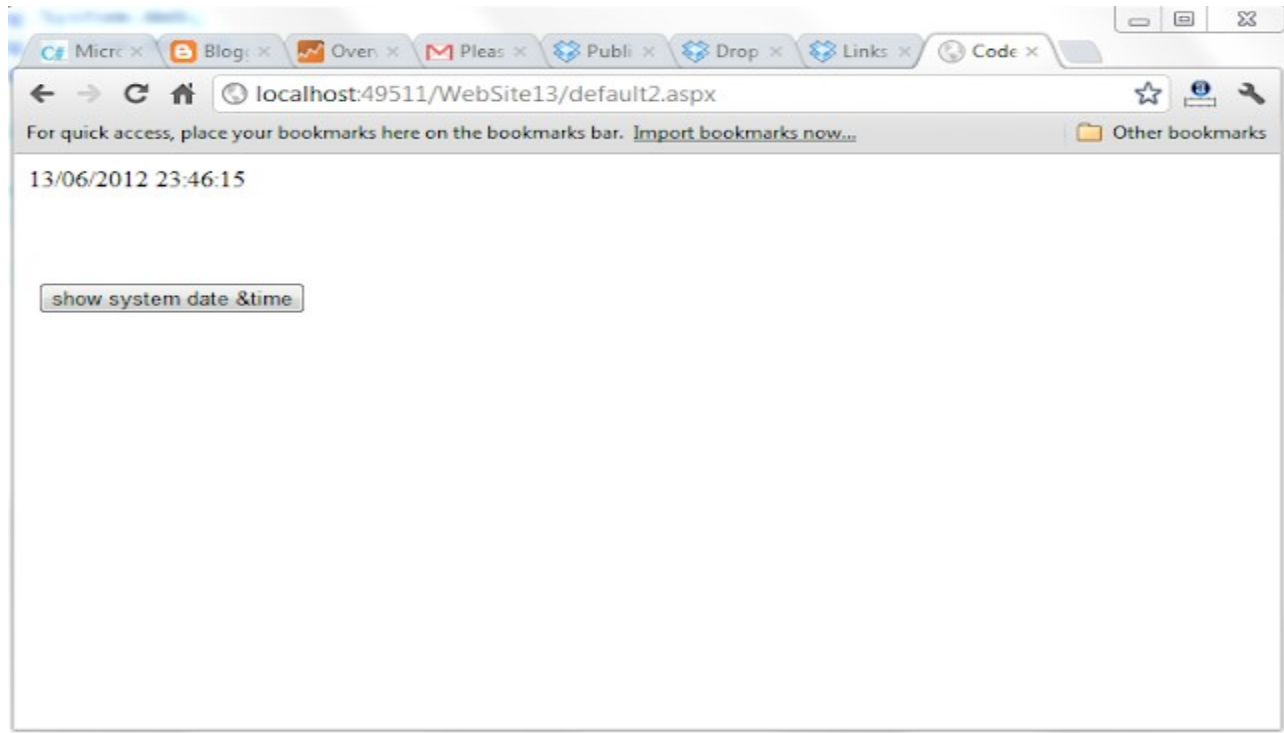


Step2: Now open the Default2.aspx.cs and paste above code. see it:-



Step3: Now run the program (press F5), Default.aspx page is open click Next Button. you will

show.
see it:->



Define In- line code and Code-behind.

In-line code:

- This code is embedded directly within the ASP.NET page.
- The code is written along side of the html page.

Code-behind:

- It refers to code for ASP.NET page which is contained within a separate class file.
- It allows a clean separation of HTML from the presentation logic.
- This code is written in a separate file and referenced by the .aspx page.

InlineCodingModel.aspx

```
<%@ Page Language="C#" %>
```

```
<!DOCTYPE html>
```

```
<script runat="server">
```

```
    protected void Button1_Click(object sender, System.EventArgs e) {
```



```

        Label1.Text = "Your City Name Is : " +
            TextBox1.Text.ToString();
    }
</script>

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title>ASP.NET inline coding model example</title>
</head>
<body style="padding:25px">
    <form id="form1" runat="server">
        <div>
            <h2 style="color:MidnightBlue; font-style:italic;">
                Inline coding model example
            </h2>
            <hr width="450" align="left" color="Gainsboro" />
            <asp:Label
                ID="Label1"
                runat="server"
                Font-Size="Larger"
                ForeColor="Crimson"
                Font-Italic="true"
                Font-Names="Comic Sans MS"
            />
            <br /><br />
            <asp:Label

```

```
ID="Label2"

runat="server"

Text="Your City"

AssociatedControlID="TextBox1"

Font-Bold="true"

ForeColor="OliveDrab"

/>

<asp:TextBox

ID="TextBox1"

runat="server"

BackColor="Pink"

Font-Bold="true"

/>

<br /><br />

<asp:Button

ID="Button1"

runat="server"

Text="Submit City"

OnClick="Button1_Click"

Font-Bold="true"

Height="45"

/>

</div>

</form>

</body>
```

</html>

Inline coding model example

Your City

Inline coding model example

Your City Name Is : KHULNA

Your City

ASP.NET Code Behind Model Example

In Code Behind model of ASP.NET the business logic place a separate file. For this example we need to create two files, one web form name Default.aspx and another file name Default.aspx.cs.

The Default.aspx holds two Label control, one TextBox control and one Button control. The all business logic need to place in Default.aspx.cs file. Here is the source code of both files.

Default.aspx

```
<%@ Page Language="C#" AutoEventWireup="true" CodeFile="Default.aspx.cs"
Inherits="_Default" %>
```

```
<!DOCTYPE html>
```

```
<html xmlns="http://www.w3.org/1999/xhtml">
```

```
<head runat="server">
```

```
    <title>Asp.Net Code Behind Model Example</title>
```

```
</head>
```

```
<body>

    <form id="form1" runat="server">

        <div>

            <asp:Label ID="Label1" runat="server" Font-Size="Large"></asp:Label>

            <br />

            <asp:Label ID="Label2" runat="server" Text="Zip Code"
AssociatedControlID="TextBox1"></asp:Label>

            <asp:TextBox ID="TextBox1" runat="server"></asp:TextBox>

            <br />

            <asp:Button ID="Button1" runat="server" Text="Submit" OnClick="Button1_Click" />

        </div>

    </form>

</body>

</html>
```

Default.aspx.cs

```
using System;

using System.Collections;

using System.Configuration;

using System.Data;

using System.Linq;

using System.Web;

using System.Web.Security;

using System.Web.UI;

using System.Web.UI.HtmlControls;
```

```
using System.Web.UI.WebControls;

using System.Web.UI.WebControls.WebParts;

using System.Xml.Linq;


public partial class _Default : System.Web.UI.Page
{
    protected void Button1_Click(object sender, System.EventArgs e) {
        Label1.Text = "Your Zip Code: " +
            TextBox1.Text.ToString();
    }
}
```

Asp.Net Code Behind Mode... x

← → ↻ ☆ http://localhost:1364/example/Default.aspx

Zip Code

Asp.Net Code Behind Mode... x

← → ↻ ☆ http://localhost:1364/example/Default.aspx

Your Zip Code: 5200

Zip Code

Difference between Single Page code and Code-behind.

Inline / Single	Code Behind
The business logic is in <script> blocks in the same .aspx file that contains the HTML and controls.	The HTML and controls are in the .aspx file, and the business logic is in a separate .aspx.cs or .aspx.vb file.
When the page is deployed, the source code is deployed along with the Web Forms page, since it is physically in the .aspx file. Though, we are not able to see the code, only the results are rendered when the page runs.	All project class files (without the .aspx file) are compiled into a .dll file, which are deployed to the server without any source code. When a request for the page is received, then an instance of the project .dll file is created and executed.
The .aspx file derives from the Page class.	The code for the page is compiled into a separate class from which the .aspx file derives.
When we write inline code we write code in the same page with Html code between scripting tags. So Each time when there is a request for page it compiles the code each time then server the page Like classic asp because inline code cannot create dll.	The code-behind approach also improved productivity (at some level) since the designer and the developer can continue working simultaneously on the same set of application. It's also easier to build & test the UI and the business logic (DLL) - separately or combined.